QoSCOLLAB : COLLABORATIVE FILTERING FOR WEB SERVICE QUALITY PREDICTION

A Project Report

submitted in partial fulfillment of the requirements

for the award of the degree of

Bachelor of Engineering

in

Computer Science & Engineering

by

13.Vishal Raju Honde

14.Saniya Salim Killedar

15.Sanket Vijay Jadhav

16.Poonam Meghraj Mundhe

under the guidance of

Dr. P.A. Patel



Department of Computer Science & Engineering

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, KOLHAPUR

YEAR: 2025-26



CERTIFICATE

This is to certify that the project report entitled **"Collaborative Filtering For Web Service Quality Prediction"** submitted by **Mr. Vishal Raju Honde(13), Miss. Saniya Salim Killedar(14), Mr. Sanket Vijay Jadhav(15), Miss. Poonam Meghraj Mundhe(16).** for the partial fulfillment of the requirement for the award of degree of **Bachelor of Engineering** in **Computer Science & Engineering** to the **Shivaji University**.

This report is record of students' team work carried out by them under my supervision and guidance.

Date:

Place: Kolhapur

Guide
Dr. P.A. Patel

HOD
Dr. R.P. Mirajkar

Ext. Examiner

Principal
Dr. V.R. Ghorpade

Acknowledgement

It is our privilege to acknowledge with deep sense of gratitude to our project guide **Dr. P.A. Patel.** for his valuable suggestions and guidance throughout our course of study and project.

We express our gratitude to **Dr. R.P. Mirajkar (HOD)** for their kind help and co-operation and special thanks to our **Principal Dr. V. R. Ghorpade** for giving us an opportunity to work on this topic.

We are highly obliged to the entire staff of the Computer Science & Engineering Department for their kind co-operation and help. We also take this opportunity to thank all our colleagues who backed our interest by giving useful suggestions and all possible help.

Mr. Vishal Raju Honde
Miss. Saniya Salim Killedar
Mr. Sanket Vijay Jadhav
Miss. Poonam Meghraj Mundhe

Abstract

In recent years, the ecosystem of web services and public APIs has grown exponentially, spanning areas like mapping, authentication, payments, messaging, storage, and specialized microservices. This diversity has created significant challenges for developers and organizations in identifying and selecting the most suitable services for their technical requirements. Traditional discovery methods, such as keyword-based searches and popularity rankings, fail to account for the personalized needs of users and the dynamic nature of service quality.

QoSCOLLAB addresses these gaps by proposing and implementing a full-stack personalized recommendation platform for web services. The system collects and manages both explicit ratings and implicit user-service interactions, training advanced collaborative filtering models to predict the likelihood of positive user engagement with specific services. By integrating real-time Quality of Service (QoS) metrics—such as response time, availability, and reliability—QoSCOLLAB delivers intelligent Top-N recommendations through an intuitive web-based interface and robust APIs. The platform further features an administrative dashboard for system analytics and ensures safe, scalable deployment suitable for production environments. Key benefits include significant reduction in decision-making time for developers, enhanced service discovery efficiency, and improved identification of high-value APIs and microservices. Additionally, the project serves as a practical demonstration of end-to-end proficiency in data engineering, machine learning model development, backend and frontend service architecture, and deployment.

QoSCOLLAB stands out for its interdisciplinary approach, merging collaborative filtering algorithms with real-time QoS monitoring, resulting in improved service reliability, productivity, and innovative contributions to both academic research and industry practice. The project lays a strong foundation for future AI-driven service management and discovery platforms capable of autonomously optimizing distributed systems.

INDEX

Sr. No.	Title	Page No.
1	**Introduction**	**1**
1.1	Need of the work	**2**
1.2	Thesis Overview	**2**
2	**Literature Survey**	**3**
3	**Proposed Work**	**4**
3.1	Formalization of the problem	**4**
3.2	System Architecture Diagram	**5**
3.3	List of modules with Description	**6**
3.4	Methodologies	**8**
4	**Software Design**	**10**
4.1	Requirement Analysis	**10**
4.2	Designed Modules	**11**
5	**Implementation and Experimental result**	**12**
5.1	Technologies used for the Implementation	**12**
5.2	Snapshots with brief description	**13**
6	**Conclusion**	**15**
7	**References**	**16**

1. Introduction

In recent years, the ecosystem of web services and public APIs has grown exponentially, encompassing domains such as mapping, authentication, payments, messaging, storage, and numerous specialized microservices. With such diversity, developers, product teams, and organizations often face challenges in selecting the most suitable services. The decision-making process is influenced by factors like quality, cost, compatibility, and user preference. While traditional methods such as keyword-based searches and popularity rankings provide some assistance, they often fail to account for the personalized needs of individual users.

To address this gap, recommender systems have emerged as a powerful solution. By analyzing user behavior, preferences, and the choices of similar users, these systems generate tailored suggestions that significantly improve service discovery.

This project proposes the design and implementation of a full-stack personalized recommendation platform for web services. The system will:

Collect and manage user-service interaction data, including both explicit ratings and implicit behavioral signals.

Train collaborative filtering (CF) models to predict the likelihood of a user engaging positively with a given service.

Deliver personalized Top-N recommendations through an intuitive web-based interface and accessible APIs.

Provide an administrative dashboard with analytics capabilities while ensuring safe, scalable, and production-friendly deployment.

Key Benefits:

Reduced decision-making time for users when selecting services.

Enhanced service discovery, enabling users to identify high-value APIs and micro-services more efficiently.

Practical demonstration of end-to-end skills, including data engineering, machine learning model development, back-end service design, front-end development, and deployment.

1.1 Need of the work

Web services and public APIs have seen exponential growth across various domains including cloud storage, payment processing, and specialized microservices. Developers and organizations face increasing complexity in selecting suitable services due to diverse quality, cost, compatibility, and real-time performance demands. Traditional approaches rely on static rankings and basic keyword searches, lacking depth in personalization and dynamic quality assessment. Intelligent recommendation systems, utilizing collaborative filtering, can streamline service selection by considering individual preferences and real-time system metrics — directly addressing gaps in current decision-making processes.

1.2 Thesis Overview

This report explains the design, development, and evaluation of QoSCOLLAB, a comprehensive recommendation platform based on collaborative filtering and Quality of Service assessment. It details literature foundations, problem formalization, advanced system architecture, module design, experimental results, and practical conclusions. The structure guides readers through motivation, technical innovations, implementation phases, and the measurable benefits of the proposed system.

2. Literature Survey/Review of Literature

Recommendation system research has evolved from early collaborative filtering (Resnick et al.) to scalable item-based algorithms (Sarwar et al.), and advanced matrix factorization (Koren et al.) for high-performance engines. Hybrid and content-based systems (Pazzani, Burke) addressed limitations in personalization versus accuracy. Quality of Service (QoS) integration matured through multi-objective optimization (Yu & Lin), standardized metrics (Zeng et al.), and robust prediction (Zheng et al.). Modern developments include neural collaborative filtering (He et al.), temporal pattern modeling (Wang et al.), and graph-based approaches (Zhang et al.). Key research gaps addressed by QoSCOLLAB are the integration of real-time QoS metrics, adaptive recommendations, and scalable distributed architectures.

3. Proposed Work

3.1 Formalization of the Problem

Develop an intelligent, scalable recommendation system capable of predicting and ranking web services based on user preferences and current QoS metrics. The system will operate in distributed microservices environments and provide developers with actionable recommendations that optimize service integration and reliability, reducing both selection time and the likelihood of operational failures.

3.2 System Architecture Diagram

QoS COLLAB's micro-services architecture has five main components:

Recommendation Engine: A suite of ML models (collaborative and hybrid) trained on user-service interaction data.

QoS Monitoring: Real-time tracking and analytics of service metrics such as response time, availability, throughput, and error rates.

User Profile Manager: Maintains identities, preferences, and history for effective personalization.

Service Registry: Database of service metadata, discoverable endpoints, and historical performance.

Analytics Dashboard: Visualization, reporting tools, and administrative controls for system evaluation.

Data layers include time-series stores for QoS metrics, graph databases for service relationships, caching for performance, and real-time streaming for continuous learning.

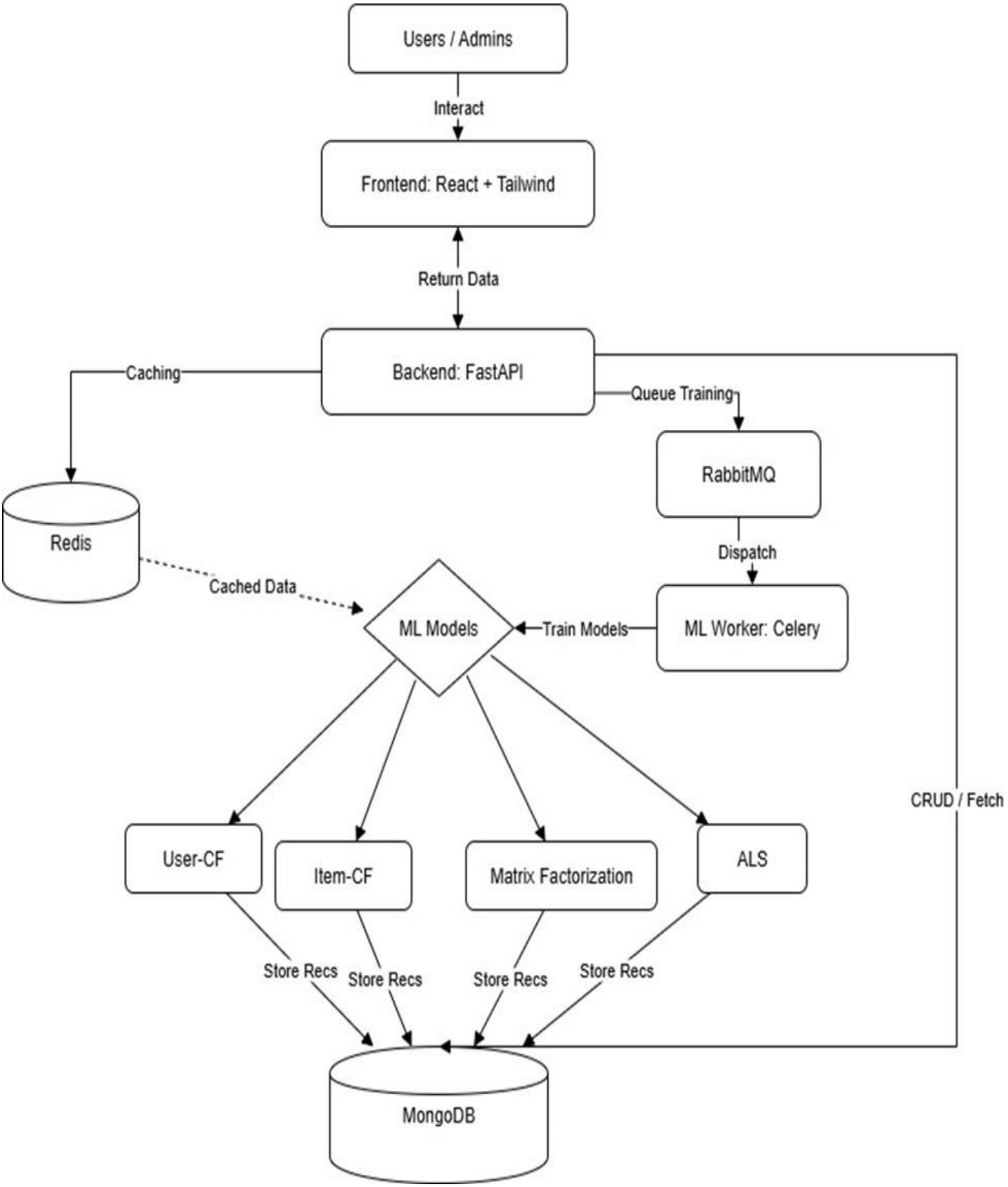


Fig 1: System Architecture Diagram.

3.3 List of modules with Description

Module 1: User Management

User Registration with JWT authentication and account management.

Role-Based Access using admin controls, permissions, and user roles.

Security Layer for encryption, data protection, and secure storage.

Module 2: Service Catalog

Service Database with metadata model, service registry, and catalog management.

Search & Filter using advanced and smart search, plus filter options.

Quality Assessment by evaluating QoS metrics, performance ratings, and service evaluation.

Module 3: ML Recommendation Engine

ML Algorithms including collaborative filtering, matrix factorization, and hybrid ML models.

Analytics Engine for usage analysis, behavior analysis, and pattern recognition.

Model Training focusing on data processing, model optimization, and performance tuning.

Module 4: Interface & Communication

Frontend Dashboard for user and admin UI, responsive design, and interactive elements.

API Middleware to handle service communication, data processing, and request handling.

Real-time Updates with WebSocket support, live data, and notifications.

Module 5: System Monitoring

Deployment Monitoring for system deployment status and uptime tracking.

Health Check Systems to monitor service health and operational checks.

Error Handling & Recovery mechanisms for error management and recovery processes.

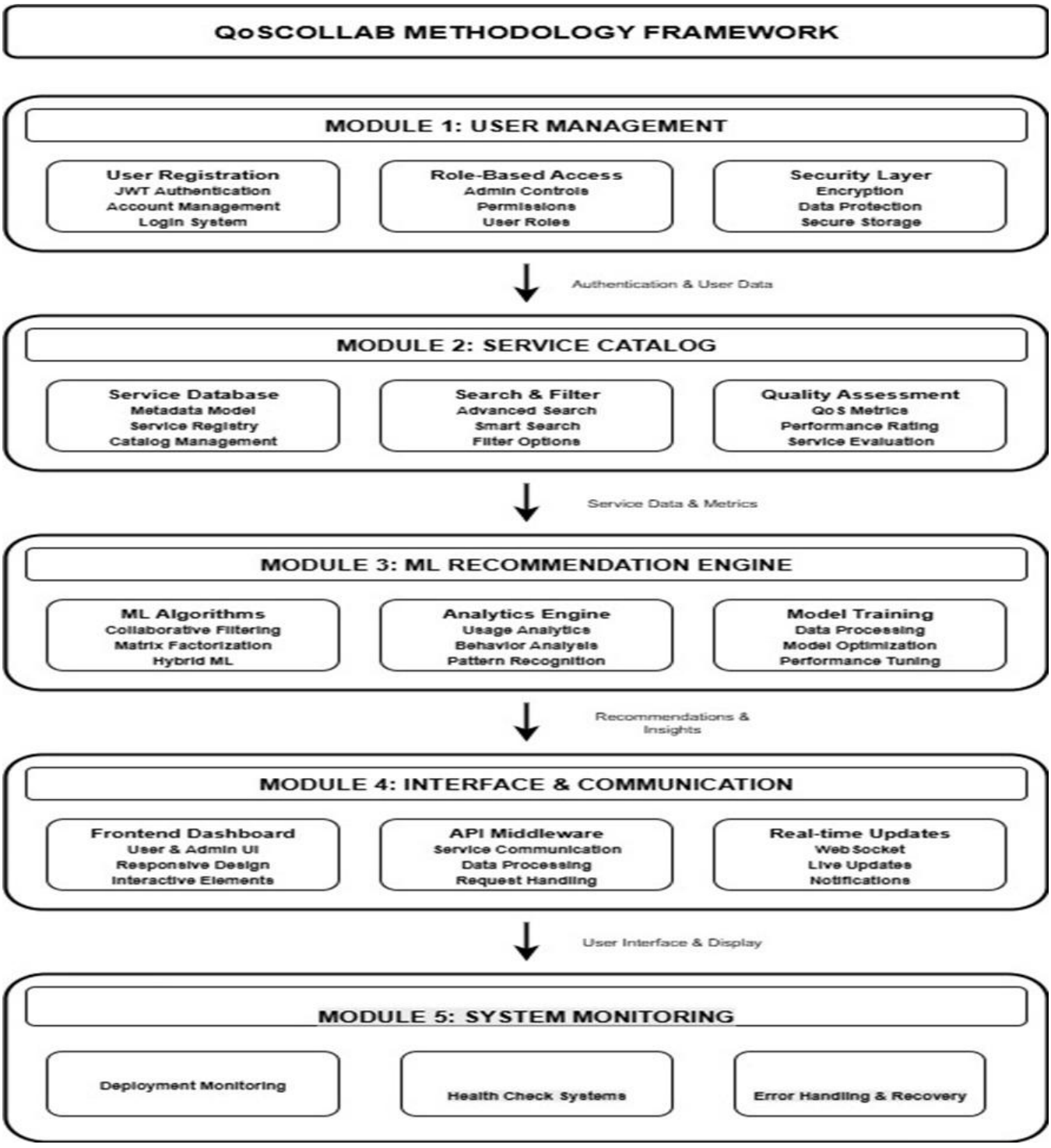


Fig:2:Collab Modules

3.4 Methodologies

The system is developed following Agile software development principles, which emphasize flexibility, modularity, and incremental improvement. The Agile methodology allows for iterative design, frequent feedback, and continuous enhancement of system components. Each development cycle (sprint) focuses on refining specific modules such as data preprocessing, model training, or evaluation, ensuring that the system evolves efficiently with each iteration.

At the core of the predictive model lie collaborative filtering algorithms, which are used to estimate missing Quality of Service (QoS) values based on known user–service interactions. Two primary collaborative filtering techniques are employed:

User-User Collaborative Filtering: This method identifies similarities between users based on their past service ratings or QoS experiences. Predictions for a target user are generated by analyzing the behaviors of users with similar patterns.

Item-Item Collaborative Filtering: Instead of comparing users, this approach identifies relationships among services (items). The system recommends or predicts QoS values for a service based on other services that share similar performance characteristics.

A Hybrid Recommender System design is also utilized, combining the strengths of traditional collaborative filtering, matrix factorization, and deep learning. This hybridization mitigates the limitations of individual methods and ensures more robust and accurate QoS predictions under both static and dynamic environments.

The data engineering pipeline is managed using Apache Kafka, which facilitates real-time data streaming, integration, and scalability. Kafka enables efficient handling of continuous QoS data updates, ensuring the system remains responsive to changing service conditions.

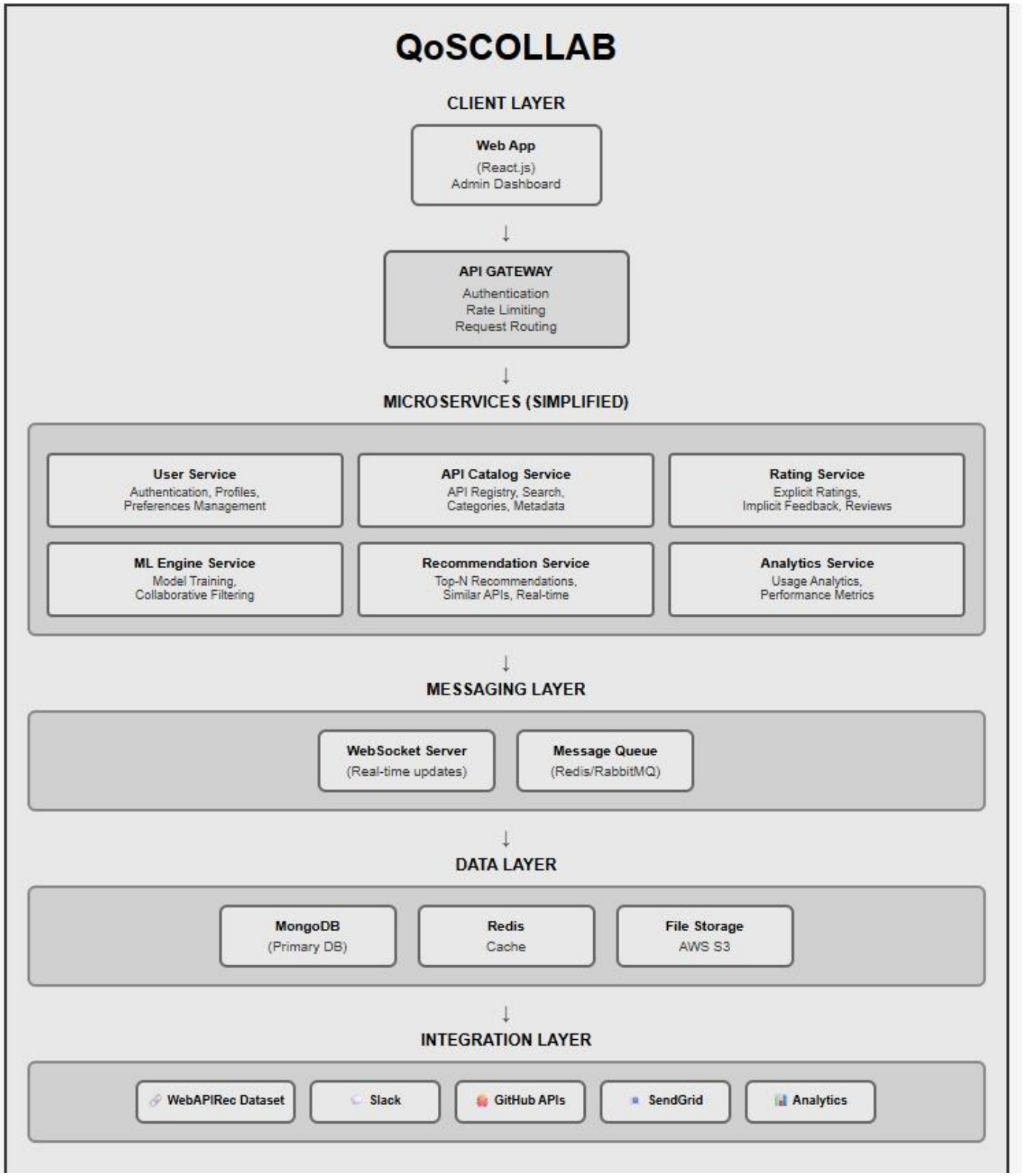


Fig3: QoSCollab

4. Software Design

4.1 Requirement analysis

4.1.1Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or higher – ensures efficient computation and model training.
- RAM: Minimum 8 GB (Recommended: 16 GB) – supports large dataset handling
- Storage: Minimum 256 GB SSD – used for storing datasets, trained models, and Kafka logs.
- GPU (Optional): NVIDIA GPU with CUDA support – accelerates deep learning and matrix factorization processes.
- Network Connectivity: Stable internet connection – required for real-time QoS data streaming, cloud access, and deployment.

4.1.2 Software Requirements

- Programming Language: Python 3.9+ – used for implementing all machine learning, data processing, and API modules.
- Frameworks and Libraries:
- Scikit-learn – for collaborative filtering and model evaluation.
- NumPy & Pandas – for numerical computations and data preprocessing.
- TensorFlow / PyTorch – for deep learning model implementation.
- Surprise Library – for matrix factorization and recommendation algorithms.
- Web Framework: Flask or FastAPI – used for backend API creation and integration.
- Database: MySQL / PostgreSQL / MongoDB – for storing QoS data and user–service interaction matrices.

4.2 Designed Module Flow Diagram

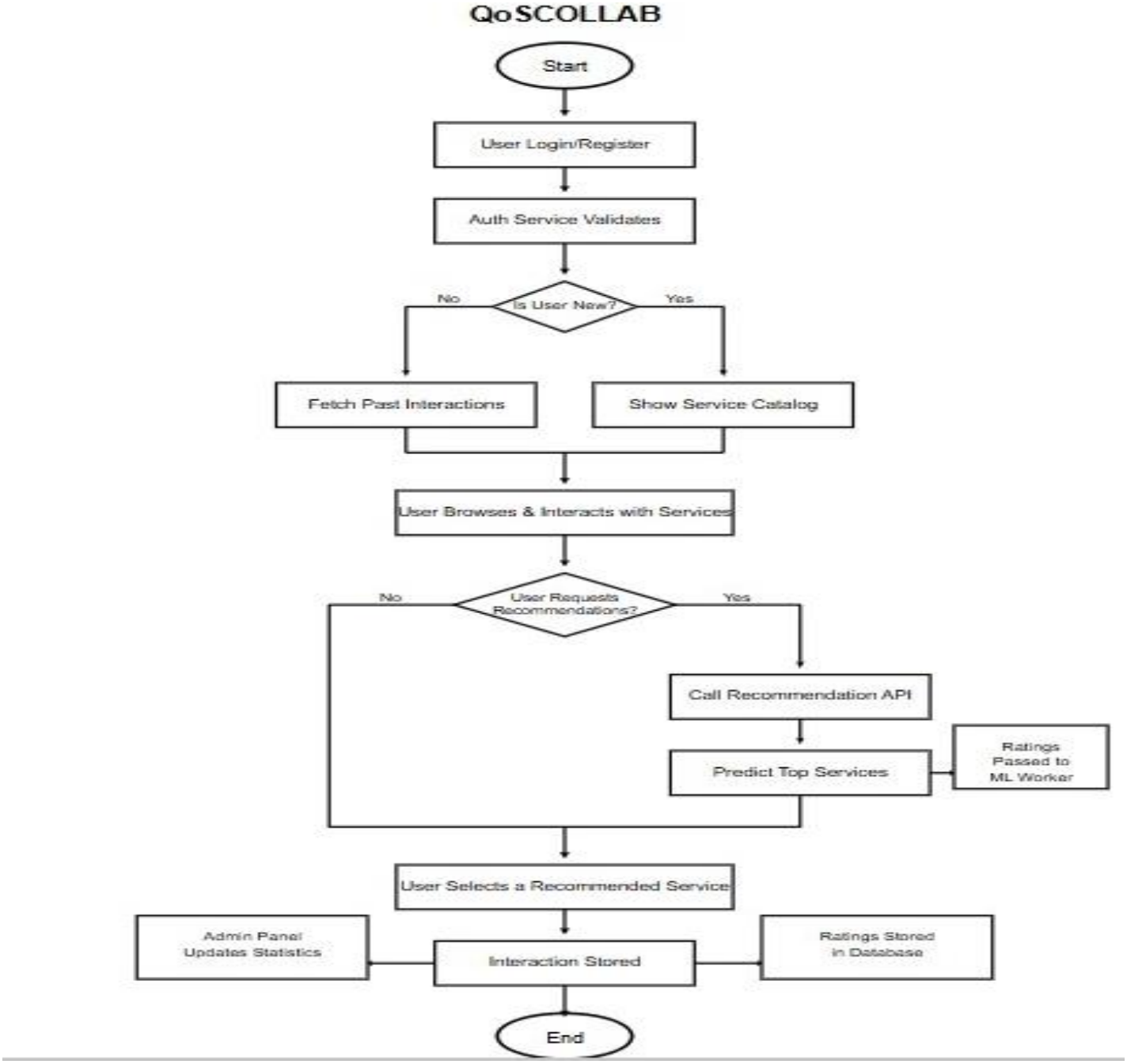


Fig-4: Data Flow Diagram

5. Implementation Details

5.1 Technologies used for the Implementation

Frontend: React.js, Tailwind CSS, real-time updates through WebSockets.

Backend: FastAPI (Python), Spring Boot (Java 17), scalable microservices.

Data Storage: MongoDB Atlas for structured data, Redis for caching, InfluxDB for time-series QoS metrics.

Machine Learning: Scikit-learn for CF models, TensorFlow for neural/hybrid models.

Streaming: Apache Kafka manages real-time data updates and model retraining events**.**

Deployment: Docker/Kubernetes for containers, Netlify for frontend, Heroku/Render for backend services**.**

Security: JWT authentication, secure REST API endpoints, validation mechanisms

5.2 Snapshots with Brief Description

Admin Panel: Graphs and analytics summarizing system usage, service performance, model accuracy, and health checks.

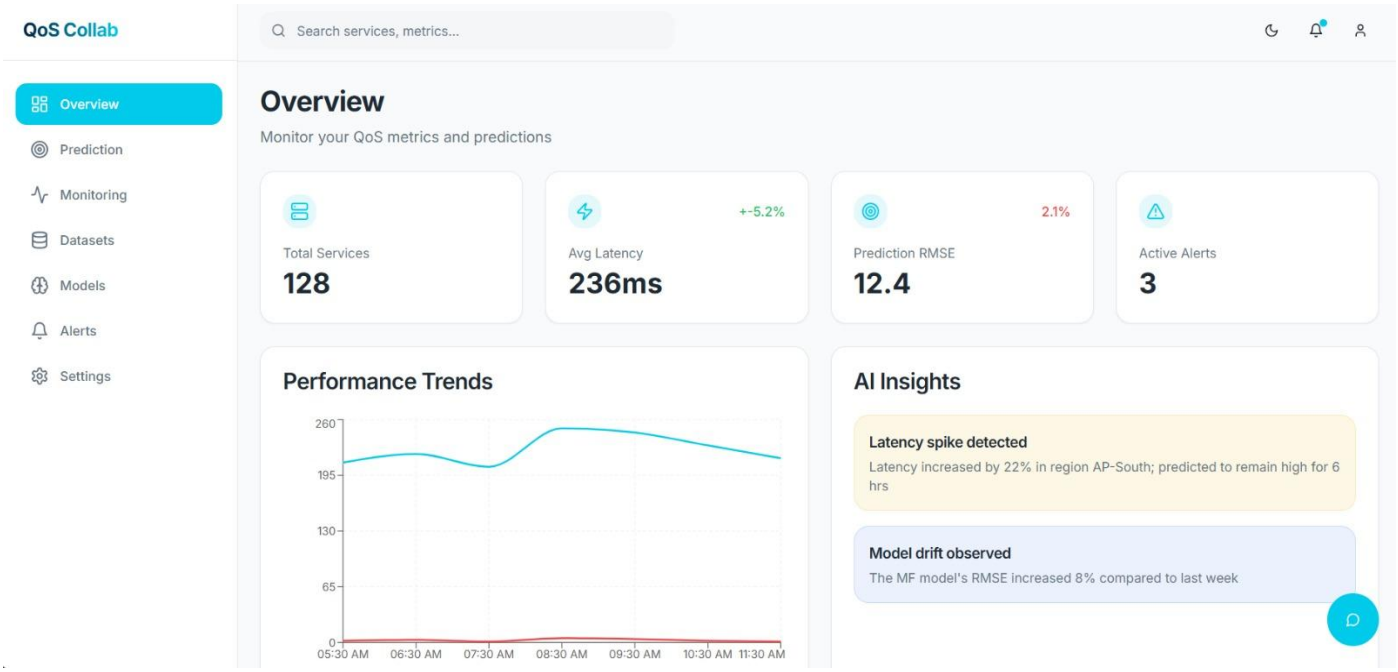
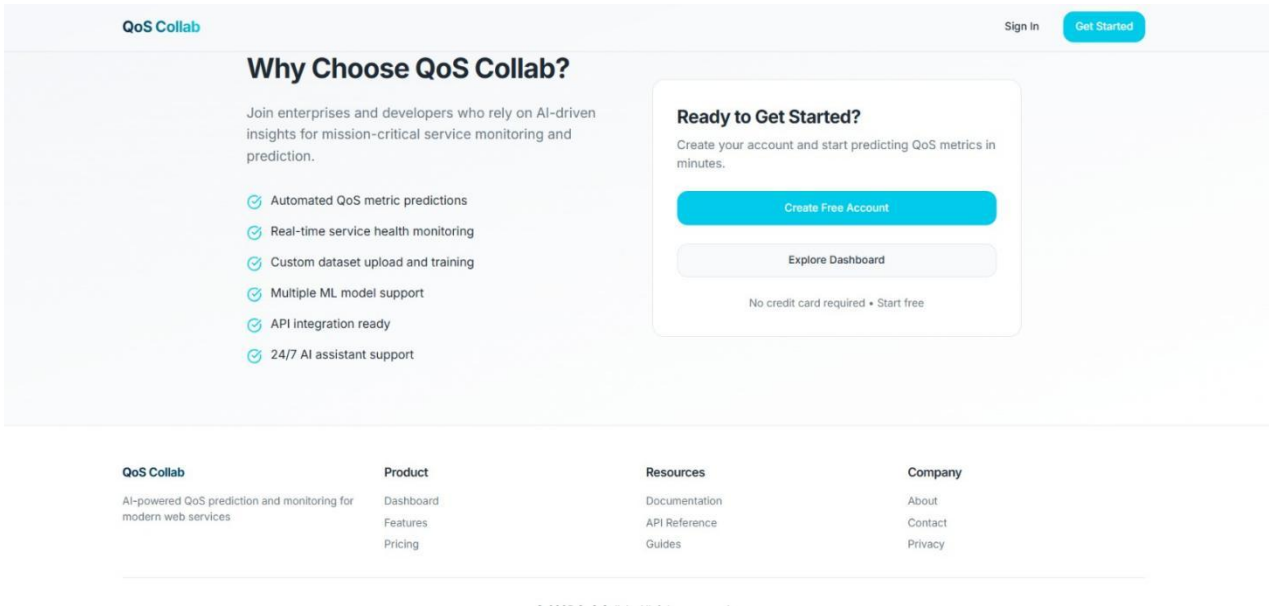


Fig 5-Admin Panel



- **Landing Page**

Fig 6:– Landing Page

- **System Health:** Real-time visualizations for service uptime, latency, and alerts on operational issues**.**

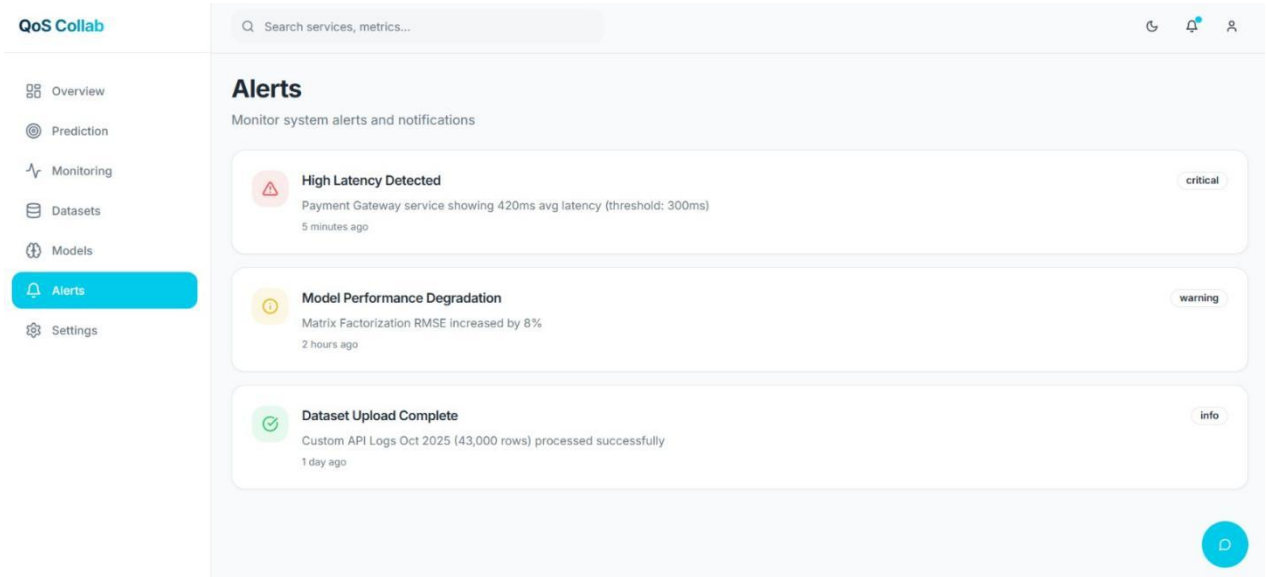


Fig.7:System Health

6. Conclusion

The QoSCollab system successfully demonstrates the application of collaborative filtering and deep learning techniques for predicting Quality of Service (QoS) values in both static and dynamic environments. By integrating user–user and item–item collaborative filtering with matrix factorization and hybrid recommendation strategies, the system effectively addresses challenges such as data sparsity and nonlinear relationships between users and services.

The inclusion of real-time data pipelines using Apache Kafka ensures scalability and adaptability, allowing the system to handle continuous QoS updates efficiently. Through iterative development under Agile principles, QoSCollab maintains modularity, flexibility, and consistent improvement across development phases.

Overall, the project achieves its primary goal of building a reliable, scalable, and intelligent QoS prediction framework, which can serve as a foundation for future research and industrial applications in web service recommendation systems. Future work may focus on enhancing model interpretability, incorporating additional contextual parameters (such as network latency or user location), and integrating reinforcement learning for adaptive service selection.

7. References

1. G. Khababa et al., "Collaborative Filtering Techniques for Predicting Web Service QoS Values in Static and Dynamic Environments: A Systematic and Thorough Analysis," in IEEE Access, vol. 13, pp. 45350-45376, 2025, doi: 10.1109/ACCESS.2025.3550284. keywords: {Quality of service;Electronic mail;Time factors;Collaborative filtering;Reliability;Prediction algorithms;Accuracy;Web services;Reviews;Heuristic algorithms;Collaborative filtering;dynamic environment;fuzzy environment;quality of service (QoS);QoS prediction;static environment;web service}.
2. Resnick, P., & Varian, H.R. (1997). Recommender systems. Communications of the ACM.
3. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms.
4. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems.
5. IEEE Computer.
6. Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. ICDM.
7. Burke, R. (2002). Hybrid recommender systems: Survey and experiments.
8. He, X. et al. (2017). Neural Collaborative Filtering.
9. Node.js Documentation.
10. React Documentation.
11. Prometheus API Documentation.
12. Google Analytics API Documentation.